# In-situ Visual Querying and Navigation of Dependency Graph Structures

JOONHO KIM, University of Toronto, Canada
KARAN SINGH, University of Toronto, Canada

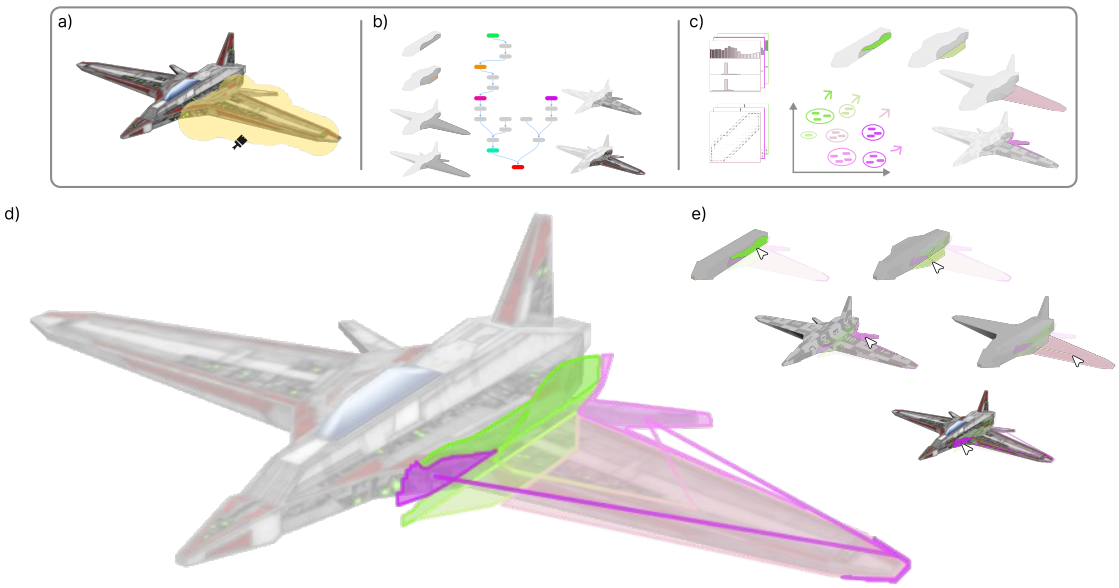



Fig. 1. Given an airplane model, we ask a visual question of how the wing changes in the dependency graph over time through highlighting the wing a). We capture masked outputs of all nodes in the dependency graph b) to cluster them on visual similarity and generate non-overlapping colored segments c). We combine these colored segments to create an in-situ visual summary of the airplane wing's dependency graph d). The summary becomes a slider interface where clicking on ticks display a summary of the cluster output and dragging between nodes in increasing gradient colors summarizes the construction history e).

Dependency graphs are a prevalent computational model for generating visual artifacts in graphics authoring systems and are often edited through an interactive graph interface. However, these graph interfaces become difficult to navigate once graphs grow in size and complexity. In particular, when users want to more generally understand how the model shape and appearance evolve, the graph interface becomes a poor search space: intermediate changes must be inspected manually, and the visual effect of a node is often not directly visible from the graph itself. We present an in-situ method for querying and exploring dependency graphs from visible consequence rather than from traditional graph-interface inspection. Users specify a region of interest by visually highlighting the rendered result in the viewport. The system then captures intermediate node outputs in image space, reduces the queried results into an ordered representative summary of visually distinct changes within that region, and constructs an in-situ slider for traversing that reduced history directly on the output. In this way, the method supports output-centric dependency-graph exploration while keeping interaction within the context of the final result.

## 1 INTRODUCTION

Dependency graphs are a central authoring paradigm in modern graphics software. In these systems, outputs are produced by evaluating a directed acyclic graph of operations that progressively derive a final visual output. The underlying dependency graph is typically exposed through a graph interface in which operations are represented as

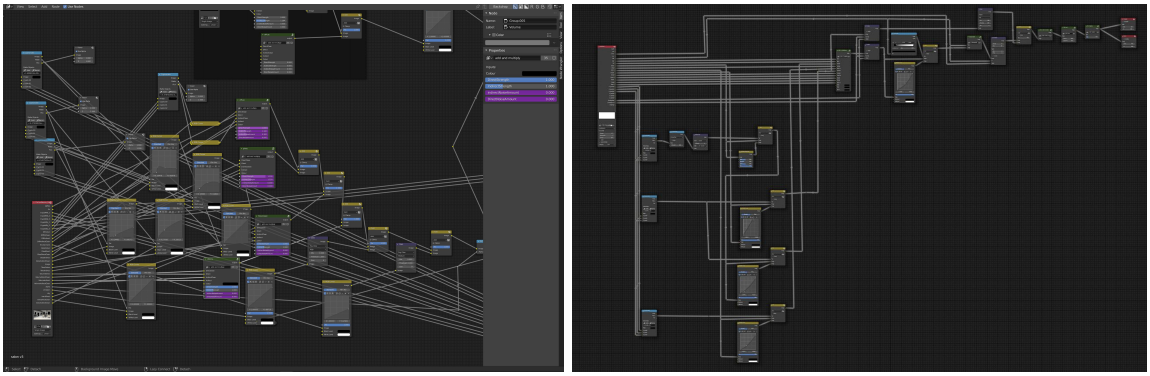


Fig. 2. Left: An example of a graph posted on a Blender community forum providing user feedback on the unreadability of large graphs. Right: An organized graph by a forum responder who suggested that the artist learn Blender's network organization features. The original artist complained it was laborious.

nodes and data dependencies are connected through directed edges. This graph-based model supports non-destructive editing, parameterization, and reuse, while graph-interface editing offers a visual programming interface that avoids some of the overhead of text-based programming [11, 13, 29]. The graph interface has become popular in industry tools such as Houdini, Maya, Blender, and Grasshopper across domains ranging from film VFX to game asset creation to architectural design.

Nonetheless, graph interfaces become harder to work with once dependency graphs grow. Production dependency graphs often contain hundreds of operators, nested subgraphs, and long chains of computation whose effect on the visual output may only be apparent later in the graph. This becomes a practical problem when artists want to perform a visual edit by finding the relevant operations or parameters in the graph to edit. Specifically, artists need to understand which operations in the dependency graph are responsible for the visible changes they observe in the final output. Yet, without prior knowledge of how the graph is organized, or which terms were used to name nodes and parameters, navigating to the right part of the dependency structure to explore the construction or edit attributes for the final outcome becomes slow.

A simple response to graph complexity is better graph organization. Modularization can break large graphs into subnetworks with narrower responsibilities, while naming conventions, layout discipline, and documentation can improve readability and navigation, especially for experienced authors working with well-maintained graphs (Figure 2). However, the effectiveness of these practices depends heavily on user expertise and sustained upkeep. They are manual to create, costly to refactor, and can become brittle as graphs evolve. More importantly, even a well-structured graph still requires the artist to translate a visible effect in the output into a structural search through the graph.

Prior work on graph visualization and visual programming helps explain this gap. Graph interfaces are effective for exposing connectivity and supporting graph-level inspection, but their readability and navigability degrade as graph size and density increase [15]. Research on graph-visualization tasks and large-graph interfaces has therefore emphasized navigation, topology inspection, and focus+context techniques for managing structural complexity [7, 24].

On the surface, the same principles of graph-visualization research can be applied to graph interfaces for dependency graphs, and are widely adopted in software. However, a crucial difference between dependency graphs and other graph visualizations is that understanding the graph structure is usually not the artist's end goal. Creating and editing

graphical outputs is the goal, and the graph interface is the typical way of controlling the underlying dependency graph. Because the creative process is focused on visible changes to the output, artists are often trying to recover which parts of the dependency graph are responsible for a particular visual effect: a silhouette change, a Boolean cut, a missing component, or a material transition. This disagreement is therefore not only one of scale, but of workflow: the user's question begins from visible consequence in the rendered result, while the interface requires a structural search through the dependency graph. In many cases that visible symptom is first noticed as a bounded phenomenon in screen space even when the responsible cause in the dependency graph is broader, more distributed, or global in its effect.

We ask the question: can dependency-graph structure be explored without beginning from the graph visualization itself? In-situ interaction can offer a useful alternative framing for this problem. Across graphics and HCI, in-situ systems place interaction and feedback in the same spatial context as the artifact being interpreted or edited, reducing mode switching and preserving perceptual continuity [17, 25, 31]. For dependency-graph exploration, that principle suggests that query specification should begin in the viewport where the user first notices a visual effect, rather than in a separate graph editor where that effect must be translated into structural querying; and navigation should be supported in the viewport on the output itself, rather than through a separate graph interface.

We present an output-centric visual-query workflow for in-situ dependency-graph exploration. Rather than beginning with the graph interface itself, the workflow begins with a highlighted region of interest in the rendered result and uses that visual query to retrieve and reduce relevant intermediate node outputs into a smaller ordered representative summary that can be traversed in the viewport. The method is therefore scoped to visual questions that can be expressed as a bounded highlighted region in the current view and for which masked differences within that region remain informative across intermediate states. In this way, this work introduces an alternative interaction model for dependency-graph exploration: users begin from visible consequence, and only later return to the graph interface once the relevant part of the history has already been reduced.

This workflow offers several benefits: users do not need to begin from detailed graph knowledge before beginning a search, the query is expressed through visible consequence rather than keywords, and the interaction remains in-situ without needing to inspect external panels. The result is a way to retrieve relevant graph states from the same visual evidence that motivated the search in the first place.

The main contributions are:

(1) an output-centric visual-query formulation for dependency-graph exploration that begins from highlighted visible consequence rather than graph-interface search;
(2) a query-driven reduction strategy that summarizes relevant intermediate node outputs into an ordered representative sequence for visual exploration;
(3) an in-situ traversal interface that lets users explore that reduced sequence directly in the scene rather than through the graph interface.

This is not a complete graph-less workflow. Our goal is to explore an alternative method for dependency-graph exploration and understanding; we do not address in-situ editing or the broader problem of provenance interfaces. More broadly, this work questions current dependency-graph workflows by emphasizing visible consequence as the first entry point into exploration rather than relying only on manual traversal of authored structure.

We begin with related work (Section 2) and then discuss the design of the interaction (Section 3). Section 4 formalizes the highlighter, clustering, and in-situ slider construction pipeline. The method is then illustrated across several dependency-graph scenarios, followed by a discussion of its scope, limitations, and future directions.

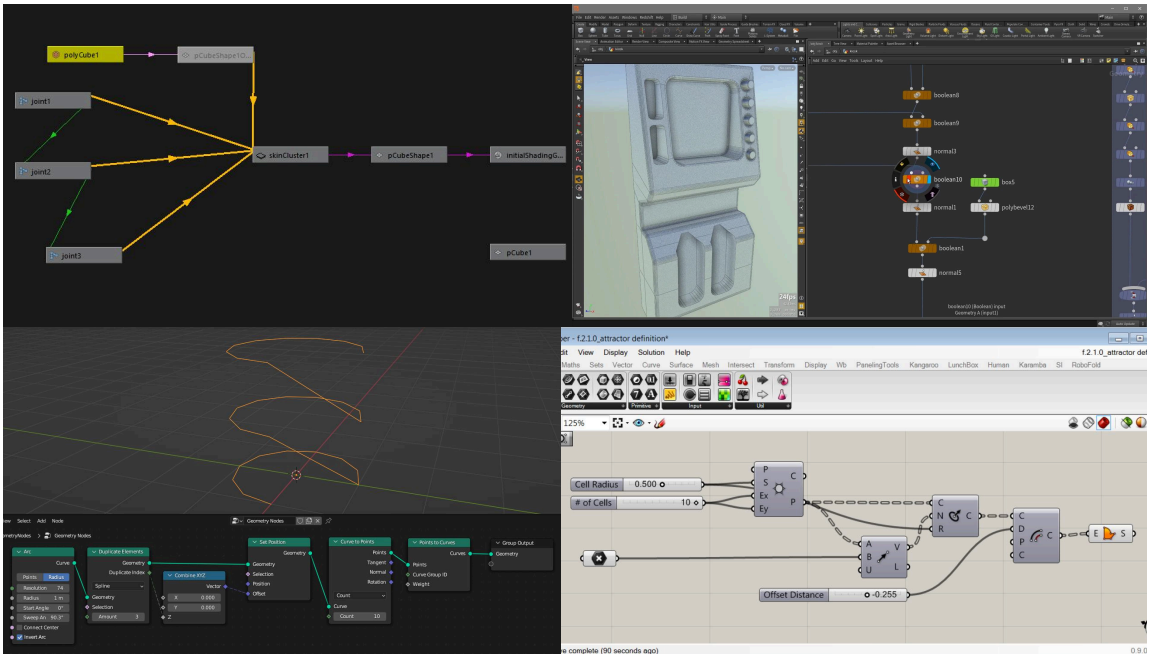



Fig. 3. Graph interfaces in commercial software. Top-Left to Bottom Right: Maya, Houdini, Blender, Grasshopper.

## 2 BACKGROUND

*Dependency graphs and graph-interface workflows.* Dependency-graph workflows emerge from a broader history of dataflow and visual programming systems, in which computation is represented as a directed graph of dependent operations [10, 11, 13, 29, 39]. In graphics authoring, this model underlies procedural shape construction, shading, and scene assembly [8, 33, 38]. Such workflows are now central to tools including Maya's Construction History, Houdini's operator networks, Blender's dependency graph (Depsgraph), and Grasshopper's visual dataflow graph (Figure 3). In these systems, the dependency graph is the computational structure that governs evaluation, while the graph is the visual interface through which that structure is authored and inspected. This representation is powerful for procedural authoring, but it does not by itself solve the problem of locating visually relevant intermediate states once graphs grow large.

*Graph and provenance interfaces.* Alongside dependency-graph representations, a parallel literature explored how users inspect, query, and revisit graph-structured information. Graph-visualization work showed that graph representations expose relationships clearly, but readability and navigability degrade as graphs grow in size and density [15]. Research on large-graph interfaces therefore emphasized overview+detail, zooming, and focus+context techniques for managing structural complexity [7], while dynamic-query interfaces showed how query and filter controls could support more fluid exploration [2]. Ware and Bobrow also showed that visual highlighting can support targeted search in graph representations by making specific relational structure easier to perceive [41]. In workflow systems, *VisTrails* treated workflow histories as provenance objects that can be versioned, revisited, and compared over time [4].

These systems are important because they support graph inspection, graph querying, and workflow provenance. However, they still assume that the user's exploration proceeds through the graph representation itself. They do not address output-centric query and traversal of intermediate states inside a dependency graph, where the user's question begins in the rendered result rather than in the graph interface.

*Design-space reduction and clustering.* Another relevant area is how large design spaces are presented to users for interaction and exploration. Marks et al. made this question explicit with *Design Galleries*, which treated a parameter space as a collection of visually browsable alternatives chosen to reveal meaningful variation [27]. This work highlighted the importance of reducing large design spaces into manageable sets of representative and comparable alternatives. Later work extended this idea from parameter studies to collections of visual artifacts. Kim et al. framed shape exploration as identifying and relating similar regions across a collection, using soft correspondences to reveal shared structure, variation, and ambiguity between 3D models while supporting interactive comparison [21]. Zhu et al.'s *AverageExplorer* frames the exploration of large image collections as navigating distributions rather than individual instances, using averages of clustered images to expose common structure, variation, and modes within the data while supporting interactive refinement of these groupings [42]. Other systems used retrieval, suggestion, or learned variability models to make large shape spaces navigable without asking users to inspect every instance. Chaudhuri and Koltun generated data-driven component suggestions for 3D modeling from a partially built shape, supporting exploratory design through retrieved examples that are relevant to the current artifact [6]. Ovsjanikov et al. learned continuous variability in shape collections and exposed that variability through a compact deformation-based navigation interface [30]. Pandey et al.'s *Juxtaform* is a closer presentation precedent for our slider construction, since it uses sparse juxtaposed abstractions from clustered shape collections to support in-context comparison [32].

These works are important because they show how large visual state spaces can be reduced into representative, comparable alternatives without destroying the distinctions a user cares about. Our setting differs in that the states being summarized are not external artifacts or unrelated samples, but intermediate outputs generated within one dependency graph. The missing capability is therefore not only browsing many alternatives, but reducing a query-specific subset of intermediate graph states into a compact sequence that can later be traversed in situ.

*Visual query by region, example, and pattern.* Content-based retrieval systems provide a more direct precedent for our output-centric query formulation. QBIC showed how users could retrieve image and video content through visual properties such as color, texture, shape, and example imagery rather than metadata alone [12]. VisualSEEk and Blobworld made the query more spatial and local: users could specify arrangements of color regions or select segmented regions to retrieve images with similar local visual structure [5, 37]. In visualization, TimeSearcher and ShapeSearch similarly let users specify desired visual patterns directly over displayed data or trendline shapes, turning a drawn region or sketched pattern into a query over many candidate outputs [16, 36]. These systems establish visual querying as a way to retrieve or filter results from visible examples, regions, or patterns. Our problem uses the same output-centric entry point, but the retrieved results are intermediate states inside one dependency graph rather than independent database items.

*In-situ and direct interaction.* Shneiderman's formulation of direct manipulation argued that interfaces become more understandable when users act on visible objects of interest rather than through detached symbolic commands [35]. Bier et al. extended that logic with *Toolglass* and Magic Lenses, showing how interface controls could be layered directly over the workspace instead of being isolated in separate panels [3]. In graphics and design applications, this idea evolved

into a broader in-situ paradigm in which interaction, guidance, and feedback are embedded in the same visual context as the piece being worked on.

From this idea, many use cases have been explored in graphics and design applications. Paczkowski et al. used in-context sketching to support architectural design directly over the site and composition being considered [31]. Lee et al.'s *ShadowDraw* embedded guidance into the drawing space itself, treating the workspace as the place where assistance should appear rather than an external control channel [25]. Igarashi et al.'s *Teddy* showed how sketch input in the drawing space can be interpreted as 3D form, while as-rigid-as-possible shape manipulation showed how direct edits to visible geometry can be propagated through an underlying deformation model [17, 18]. Our face-rig UI work is a closer example for the control-layout aspect of this chapter, since it places controls near the facial regions affected by high-dimensional rig parameters [20].

Other graphics work has explored these principles specifically for DAG-based representations. Michel et al. used DAG amendment to support inverse control of parametric shapes while keeping the underlying procedural structure intact [28].

These works establish why in-situ interaction is valuable once the user already knows what visible object or control should be manipulated. Our problem begins earlier: the user first needs to query and recover which intermediate states inside a dependency graph are relevant to a visible effect in the final result. This makes output-centric, in-situ querying an appropriate direction: the query can begin in the desired region, and the resulting reduced history can remain anchored in the same spatial context rather than forcing repeated transitions to a detached node editor.

## 3 DESIGN AND OVERVIEW

### 3.1 Graph-interface Workflow

Dependency-graph workflows in graphics software are typically split across a scene view, a graph editor, and a parameter editor. Artists author visual artifacts by chaining node operations together within the graph editor, adjusting individual node parameters in the parameter editor, and visualizing the current result in the scene view. Typical workflows cycle through these main panels: artists imagine a desirable visual edit, find existing or create new nodes to support this edit, and tweak parameters to achieve the desired result in an iterative process.

As dependency graphs grow, finding the relevant node becomes a bottleneck in that workflow. Large networks contain nested subnetworks and long chains of computation whose visible effects only become clear much later in evaluation. Though the graph remains useful for explicit authoring, it can become a slow search space for answering visual questions because it does not directly reveal which nodes are involved in the desired edit quickly. Some graph interfaces allow rendering small thumbnails on nodes themselves or allow display nodes and visibility toggles to view the output of specific intermediate nodes, but these features are still set within graph interfaces themselves.

This workflow forces artists to carry two mental models at once: a visual model of the graphical output in the viewport and a structural model of the dependency graph in the editor. In the previous section, we reviewed different methods of facilitating graph navigation and inspection, but they all assume that the artist is trying to understand the graph itself as a primary task. However, an artist who notices a seam, a protrusion, or a missing component is not trying to understand the graph structure for its own sake; they are trying to recover the cause of a visible symptom in the result. As navigation becomes increasingly difficult as graphs scale, maintaining that structural model during early search becomes a burden.

We therefore defer graph-interface structural details during early exploration and provide a more output-centric workflow. We say this specifically for exploration tasks; graph interfaces are currently necessary for authoring new chains of operations. The tool we propose is not meant to replace the graph interface but to complement it by providing a more visual entry point for exploration.

### 3.2 Design Principles

A visual question from the artist about the output of a construction history implies an informal in-situ interaction between the artist and the rendered artifact in the viewport. This provides an opportunity for a formal entry point into exploration and navigation. An in-situ interaction requires a visual and localized anchoring of the interaction and visual feedback that is directly related to the artifact in question, which complements the fact that the artist is already focused on the artifact. Additionally, in-situ interaction benefits from direct manipulation as an interaction style. As the artist's attention shifts into the scene, the exploration of the dependency graph should also stay within the scene as much as possible, rather than forcing the artist to switch back and forth between the scene and the graph interface. Direct manipulation, particularly scrubbing, is a universally familiar interaction style that drives inspection and navigation, especially for temporal sequences (videos). Because construction histories are ordered sequences of operations, scrubbing through a sequence of intermediate states is an appropriate candidate method of exploring the history and understanding how the artifact is formed.

The following design principles guide an in-situ method for exploring dependency graphs while deferring graph structure during early inspection.

*D1: Query through visual consequence.* Artists usually begin with a visible issue or a desired visual result before knowing which node operation produced it. From this querying task, not every node in the dependency graph is relevant to that question and needs to be searched. Our first principle is to query through visible consequence from the artist's view rather than direct lookup in the graph interface. Instead of asking artists to search for the right operator name or network region, the tool lets them specify a region of interest directly in the viewport and retrieves intermediate node outputs whose rendered results exhibit relevant variation within that selected region. This does not assume that the responsible node is itself spatially local in graph terms; it assumes only that the visual question can be expressed as a bounded highlighted region in the current view.

*D2: Image-space analysis.* We treat image space and the artist's current view as the primary domain for estimating visual relevance across different node outputs in the dependency graph rather than analyzing the graphical representations themselves. The underlying graphical representation may be geometry, point clouds, signed distance fields, simulations, or learned outputs, which poses compatibility challenges for direct graph analysis. But all representations are visually rendered in screen space, and the screen-space domain remains the most direct representation of what appears, disappears, overlaps, or changes from the artist's point of view. This also motivates persisting the visual query in a view-aware form, so the queried region can remain meaningful under modest camera change while the search still begins from the artist's current view. The method is strongest when masked differences inside that highlighted region remain discriminative across intermediate states, even if the underlying graph dependency is more distributed.

*D3: Reduced history traversal in place.* Even after a visual query narrows the search, the remaining nodes are still too large to inspect one by one. The interface should therefore summarize these states to a smaller set of distinct representatives and let the artist traverse them without leaving the viewport. We therefore reduce the relevant

intermediate node outputs into a smaller ordered representative sequence and realize that sequence as an in-situ query slider in the viewport. In the current implementation, that slider is not formed by directly replaying every representative state as an attached control point. Instead, the ordered sequence is converted into a smaller set of image-space segment cues, and only the winning cleaned path through those cues is realized as the final in-situ slider. This keeps navigation embedded in the scene, reduces context switching, and supports provenance inspection within the selected region without requiring continuous returns to the node editor.

### 3.3 Overview

The tool follows a three-step workflow: *query*, *reduction*, *and in-situ traversal*. Consider an artist who wants to understand how the wing of an airplane is formed in an unfamiliar dependency graph (Figure 1). Instead of switching to the graph editor and tracing nodes manually, the artist highlights the wing as a region of interest directly in the viewport and issues a visual query.

In the first step, the tool captures the queried portions of intermediate node outputs from the current view. It then reduces those queried results into a smaller ordered representative summary of the major visual changes within the selected region.

Finally, the tool presents this reduced sequence through an in-situ query slider embedded in the scene. The slider is built from localized segment candidates extracted from the ordered representative sequence in image space, cleaned into one non-overlapping path, and then attached back to the current scene for traversal. The artist can traverse the sequence directly in the viewport to understand a compact visual summary of how the selected region changes through the dependency graph. This slider allows the user to keep attention on the artifact itself rather than on a separate graph-interface view. In this way, the workflow begins from visible consequence and only returns to the graph interface after the search space has already been reduced for editing purposes.

## 4 METHOD

Given an active dependency graph, a current camera, and an artist-specified visual query, our goal is to recover a small ordered representative sequence of node outputs that are visually relevant to the query, visually distinct from one another, and traversable in situ. The method takes as input

$$I = (G, v_0, \mathbf{C}, \mathbf{M}_q), \tag{1}$$

where $G$ is the active dependency graph, $v_0$ is the currently displayed baseline node, $\mathbf{C}$ is the active camera, and $\mathbf{M}_q$ is the screen-space visual-query mask drawn by the artist. The output is

$$O = (\mathcal{C}, \boldsymbol{\pi}, \mathcal{P}^\star), \tag{2}$$

where $\mathcal{C} = \{C_1, \ldots, C_K\}$ is clusters of nodes in $G$ with visually similar outputs within the masked region, $\boldsymbol{\pi} = (v_1^\star, \ldots, v_K^\star)$ is the ordered sequence of representative nodes for slider tick order, and $\mathcal{P}^\star$ is the selected cleaned path used to build the final slider. The final query slider is a derived in-situ object realized from this representative order and selected path.

We model the dependency graph as a directed graph $G = (\mathcal{V}, \mathcal{E})$ where each node $v \in \mathcal{V}$ is an operator with inputs and outputs and each directed edge $(u, v) \in \mathcal{E}$ feeds output from $u$ into $v$. Let $\mathcal{X}(v)$ denote the scene state obtained after evaluating the dependency graph up to node $v$. Under the active camera $\mathbf{C}$, the rendered output of node $v$ is an image

$$\mathbf{I}_v = \mathrm{Render}(\mathcal{X}(v); \mathbf{C}), \tag{3}$$

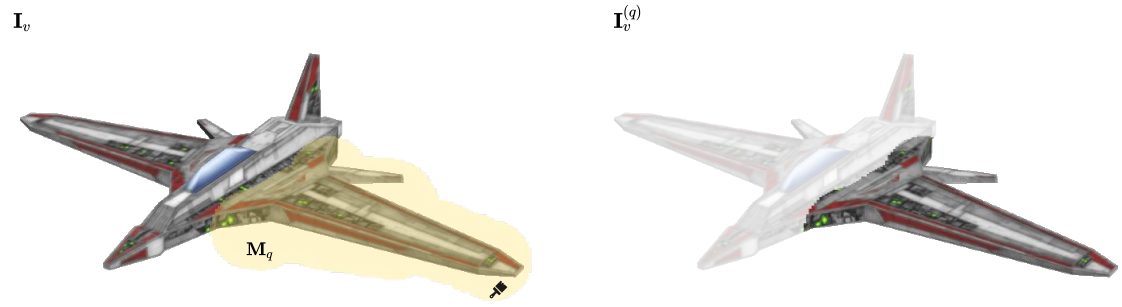



Fig. 4. The user highlights to create a mask $\mathbf{M}_q$ over image $\mathbf{I}_v$ to create a masked image $\mathbf{I}_v^{(q)}$ that retains only the queried region. The masked out region is displayed as reference.

where Render denotes the rendering operator.

### 4.1 Visual Querying

*Highlighter brush.* The artist highlights a region of interest directly in the viewport using a highlighter brush to ask how that region changes across the dependency graph (Figure 4). The brush is drawn over the currently displayed output $\mathbf{I}_{v_0}$, so the query mask is anchored to the active view of node $v_0$. Let $\mathbf{M}_q : \Omega_{\text{img}} \to [0, 1]$ denote a binary query mask over the image domain $\Omega_{\text{img}}$ where 1 includes the pixel in the image and 0 excludes the pixel from the query. Highlighting is a common method of annotation that keeps the underlying view visible, and this method is extremely cheap and expressive for artists.

Although the query is specified in screen space, the tool preserves a lightweight 3D-backed support for the painted region so the query can survive modest camera changes. In practice, the stroke is projected onto the viewed artifact and stored as support samples that can be reprojected back into the current view to recover the active query mask.

Once a query is requested, the tool traverses the dependency graph $G$ and renders each intermediate node output $\mathbf{I}_v$. From these outputs, only the queried region is retained. We define the masked node output

$$\mathbf{I}_v^{(q)} = \mathbf{M}_q \odot \mathbf{I}_v, \tag{4}$$

where $\odot$ denotes element-wise masking.

### 4.2 Clustering

With the set of masked node outputs $\mathbf{I}_v^{(q)}$, our goal is to cluster queried nodes by visual similarity within the highlighted image region and reduce them to a smaller set of distinct outputs. Figure 5 shows the resulting grouping on the airplane example: visually similar masked outputs are grouped into clusters, and each cluster contributes a representative node for the reduced sequence.

*Combined LAB+HOG feature clustering.* Let $\mathcal{V}_q \subseteq \mathcal{V}$ denote the subset of dependency-graph nodes whose queried outputs yield valid masked measurements. For each queried node output, we compute an image descriptor that captures color and structure from the masked images. Specifically, we use a combined LAB color feature plus Histogram of Oriented Gradients (HOG) feature. The LAB component summarizes color and material variation in the queried region

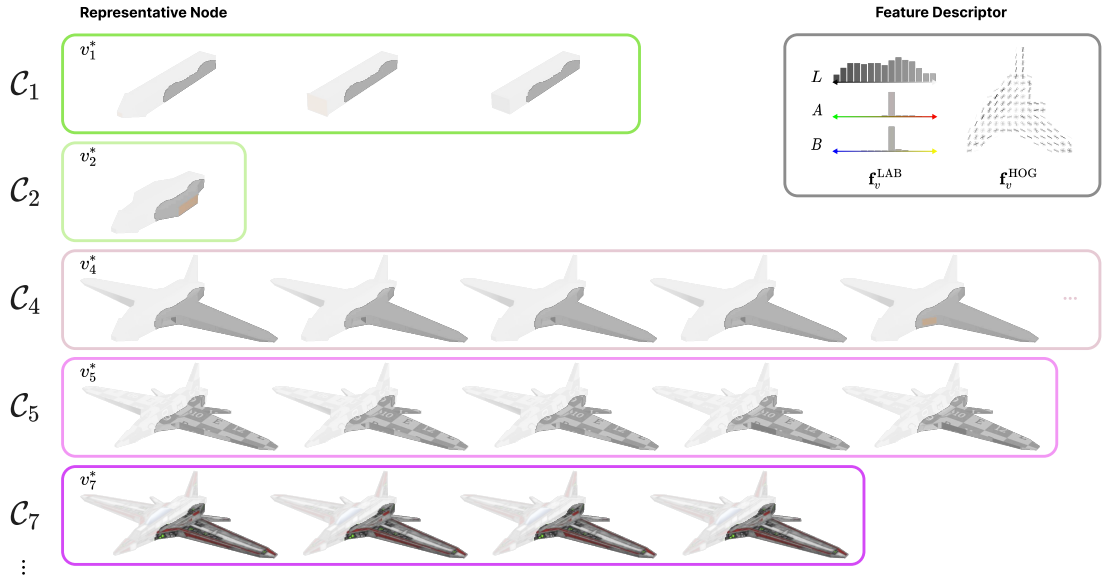



Fig. 5. Clusters of nodes from the airplane example show visually similar outputs within the query region. The feature descriptor used for clustering captures both color and structural characteristics.

using color histograms in LAB space [19], while the HOG component summarizes coarse structural variation using local image gradient orientations [9]. We write these two feature families as

$$\mathbf{f}_v^{\text{LAB}} = \phi_{\text{LAB}}(\mathbf{I}_v^{(q)}) \in \mathbb{R}^{d_{\text{LAB}}}, \tag{5}$$

$$\mathbf{f}_v^{\text{HOG}} = \phi_{\text{HOG}}(\mathbf{I}_v^{(q)}) \in \mathbb{R}^{d_{\text{HOG}}}, \tag{6}$$

where $d_{\text{LAB}} = 96$ for the concatenated 32-bin histograms over the $L$, $A$, and $B$ channels, and $d_{\text{HOG}} = 7200$ for the HOG descriptor extracted from the resized masked crop. We then concatenate them into one clustering descriptor

$$\mathbf{f}_v = \left[\mathbf{f}_v^{\text{LAB}}\,;\,\mathbf{f}_v^{\text{HOG}}\right] \in \mathbb{R}^{d_{\text{LAB}}+d_{\text{HOG}}}. \tag{7}$$

Using these descriptors, we then cluster the queried nodes:

$$C = \text{Cluster}\Big(\{\mathbf{f}_v\}_{v\in\mathcal{V}_q}\Big), \tag{8}$$

where $C = \{C_1, \ldots, C_K\}$ is the resulting partition of queried nodes into visually similar groups in the combined LAB+HOG feature space. Other image representations could be used to segment, label, or cluster the masked regions before reduction. Classical methods such as graph cuts or superpixels could produce boundary-aware regions [1, 23], while neural methods such as STEGO and SAM could provide semantic segmentation labels or promptable masks [14, 22]. We use LAB+HOG as a lightweight alternative, prioritizing fast processing over semantic accuracy. For clustering, we explored both $k$-means and agglomerative clustering over these combined descriptors seeing similar results in cluster quality [26, 40]. The number of clusters is selected from a small sweep $K \in \{4, 5, 6, 7, 8\}$ whenever the queried graph is large enough, and the best silhouette-scored configuration is retained [34].

*Representative node selection.* For each cluster $C_k$, the tool chooses the medoid of the cluster as the representative node $v_k^\star$ whose masked output is most visually central to the rest of that cluster. We write this selection as

$$v_k^\star = \arg\min_{v \in C_k} \sum_{u \in C_k \setminus \{v\}} \delta\left(\mathbf{I}_v^{(q)}, \mathbf{I}_u^{(q)}\right), \tag{9}$$

where $\delta$ denotes pairwise visual difference between two masked outputs in the same cluster. In other words, the representative is the node whose queried image is closest, on average, to the other queried images in that cluster. These representative nodes become the characteristic states that we explore.

### 4.3 In-Situ Slider Creation

To explore the representative nodes, we construct an in-situ slider from the ordered representative sequence $\boldsymbol{\pi}$. A slider requires ticks to apply set configurations and a path to connect those ticks into a traversable control. We choose to use visual features of the representative outputs to form our slider ticks rather than plan circles or lines and use $\boldsymbol{\pi}$ to define the chronology of those ticks. The tick shapes provide visual cues about the underlying node outputs, while their chronology provides a semantic ordering that reflects the underlying dependency structure.

Our path is the ordered sequence of representative nodes according to the chronology induced by their dependency-graph positions. This yields the ordered representative sequence

$$\boldsymbol{\pi} = (v_1^\star, \ldots, v_K^\star), \tag{10}$$

where representatives further upstream appear earlier in the sequence and representatives closer to the final displayed result appear later.

*Difference-guided segment candidates.* Designing the shape of the slider ticks is a key challenge because our goal is to create an in-situ control that visually summarizes the underlying node outputs while remaining legible and interactive. Figure 6 shows the pipeline used to derive these tick candidates from adjacent representative outputs: we compute masked differences, threshold high-signal regions, and segment them into candidate tick shapes.

Because the representative sequence $\boldsymbol{\pi}$ provides a chronological ordering of representative nodes, we use differences between neighboring representatives to identify local visual changes. For each adjacent pair in the ordered sequence, we define a difference image

$$\mathbf{D}_{k,k+1}^\star = \Delta\left(\mathbf{I}_{v_{k+1}^\star}^{(q)}, \mathbf{I}_{v_k^\star}^{(q)}\right), \tag{11}$$

derived from the ordered pair $(v_k^\star, v_{k+1}^\star)$. These comparisons are evaluated in LAB image space over the queried region. The resulting difference images display high signal where neighboring representatives differ and low signal where they remain visually similar. These high signal regions can be used as slider tick shapes for either $v_k^\star$ or $v_{k+1}^\star$ because they represent visual change that occurs between those two states. We assign these difference image components to one of the two neighboring representatives so each representative owns a difference image.

Each difference image is normalized and segmented into connected components using thresholding followed by connected-component extraction. This produces regions $r_k$ that can be used as slider tick shapes.

Let

$$\mathcal{R}_k = \{r_k^1, \ldots, r_k^N\} \tag{12}$$

denote the ranked owned segment candidates assigned to representative $v_k^\star$. Representatives that receive no owned segment are treated as unassigned and are handled with the same fallback candidate-generation path as other missing-support cases. In the implementation, we keep the top $N = 2$ candidates per representative.

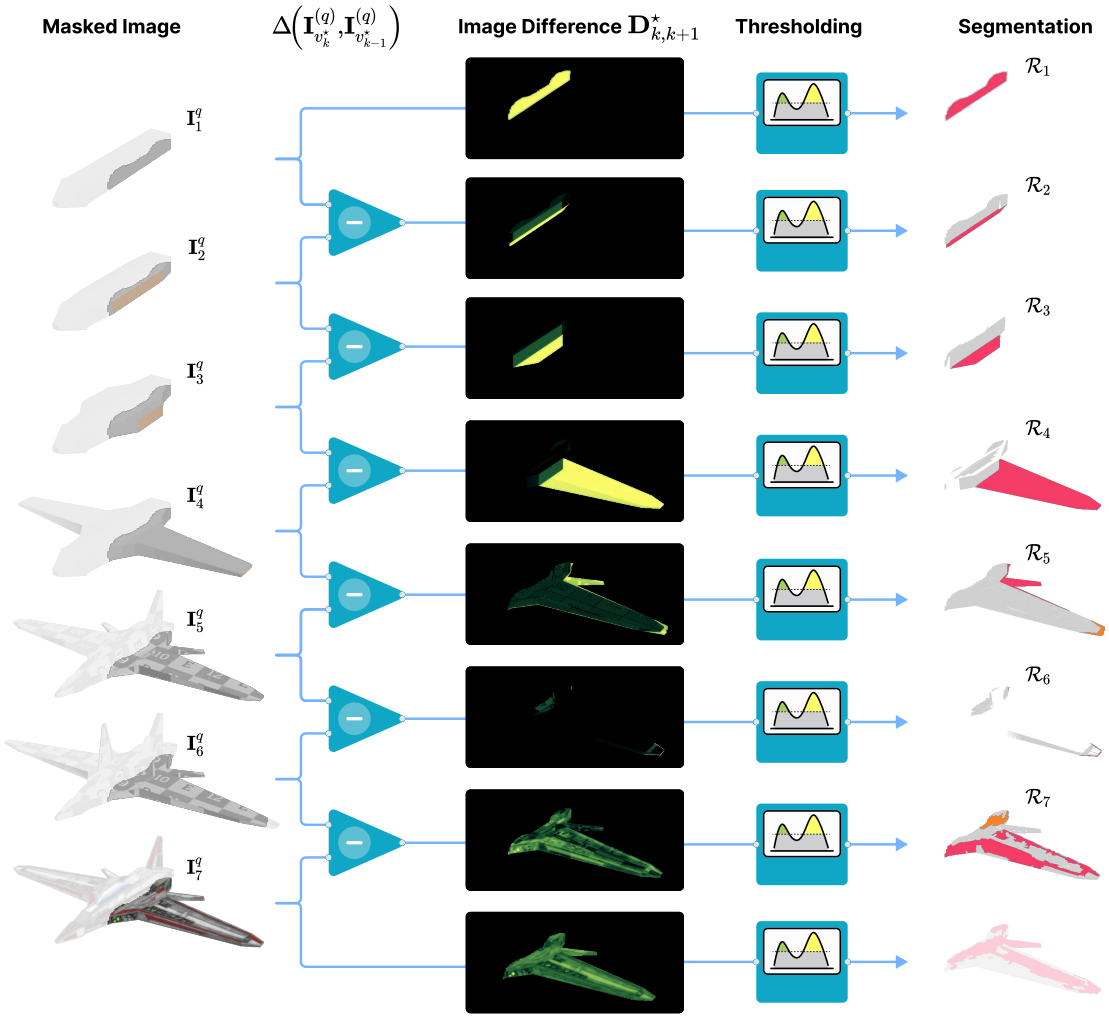



Fig. 6. Adjacent representative outputs $(\mathbf{I}^{(q)}_{v^\star_{k+1}}, \mathbf{I}^{(q)}_{v^\star_k})$ are used to compute difference images $\mathbf{D}^\star_{k,k+1}$ in LAB space. High-signal regions that indicate local visual changes between neighboring representatives are retained through a thresholding pass. These regions are segmented into connected components $\mathcal{R}_k$ (with segments colored in red and orange), and each component is assigned to one of the neighboring representative that best explains it. The resulting owned segments become candidate tick shapes for the in-situ slider. In this figure, the last segmentation is not assigned.

*Candidate-path search across ordered clusters and winning-path selection.* The local image-space candidates do not yet define the final slider because each cluster can contribute several plausible segment regions. We therefore search over

candidate paths that choose one raw marker candidate per ordered cluster:

$$\mathcal{P} = \mathcal{R}_1 \times \mathcal{R}_2 \times \cdots \times \mathcal{R}_K. \tag{13}$$

The result is a set of ordered candidate paths, each of which represents one possible slider interpretation through the representative sequence.

For a candidate path $\mathcal{P}_i$, its segments $r_k^j$ may still visually overlap one another, which would produce ambiguous slider ticks. We therefore clean each candidate path by greedily resolving overlapping segment regions until the selected supports no longer overlap. The cleanup prioritizes overlaps between nearby representatives in the ordered sequence and assigns contested pixels to the stronger segment, preferring owned difference segments over fallback segments. This produces a non-overlapping set of tick supports while preserving as much usable area as possible for each representative.

Let $\widetilde{r}_k$ denote the cleaned segment assigned to ordered cluster $k$ for a candidate path. The cleaned path is then

$$\widetilde{\mathcal{P}} = (\widetilde{r}_1, \ldots, \widetilde{r}_k). \tag{14}$$

Once all candidate paths have been cleaned, we optimize for a single winning cleaned path. This choice is driven by the usability of the resulting segment ticks. For each cleaned path step, the implementation computes a tick-size score from the surviving cleaned region. The winning path is the path with the largest minimum tick area across all steps, so the weakest tick on the path is as large as possible.

The winning path is then selected lexicographically:

$$\mathcal{P}^\star = \arg\max_{\widetilde{\mathcal{P}}} \left( \min_k area(\widetilde{r}_k) \right), \tag{15}$$

The slider is created by connecting pairs of cleaned segments $(\widetilde{r}_{k-1}, \widetilde{r}_k)$ with straight line segments and a slider thumb to scrub along the path. Activating the slider tick displays the corresponding representative node output and highlights $\widetilde{r}_k$ as well as paths connected to $\widetilde{r}_{k-1}$ and $\widetilde{r}_{k+1}$. Segments are colored in a gradient to indicate cluster progression with the most saturated segments marking the end ticks of the slider.

*Implementation.* The tool is implemented inside Houdini using Python and Houdini Viewer States, with the viewport used as the primary query surface and the dependency graph used as the computational structure being searched. The image-space stages of the pipeline use the Python libraries NumPy, SciPy, scikit-image for feature extraction and segmentation, scikit-learn for clustering, and OpenCV for other image processing steps. We run this on an Apple M3 MacBook Pro.

For our in-situ slider in 3D, we project the path $\mathcal{P}^\star$ onto the Houdini geometry. Specifically, each cleaned region $\widetilde{r}_k$ is raycast back into the world and attached to the geometry of the representative node $v_k^\star$ (or the previous node $v_{k-1}^\star$ in case the visual difference between $(v_{k-1}^\star, v_k^\star)$ resulted in lost area), so the slider remains attached to the artifact as the user moves through the scene.

## 5 EXAMPLES

We explored several dependency graphs from the Houdini community to test the tool. These examples show how different procedural networks respond to visual queries from the viewport. Some paths are omitted in the figures to emphasize segment shapes.

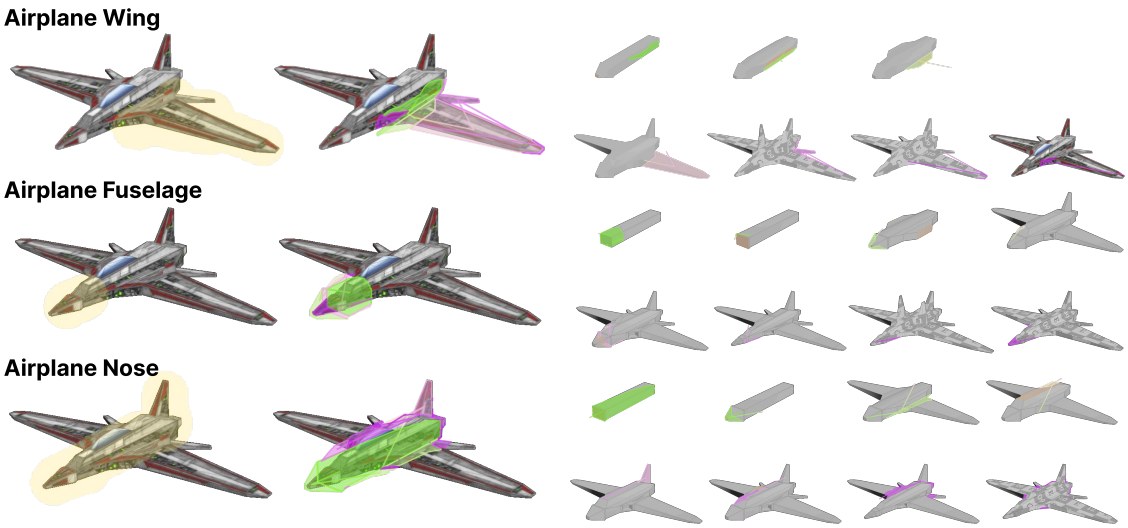



Fig. 7. Different visual queries on the same airplane recover different local construction histories. Querying the wing emphasizes states that form the wing surface and supporting outlines, while querying the fuselage or nose shifts the representative sequence and segments toward the body and front. The same dependency graph can therefore yield several query-specific construction-history summaries depending on where the artist asks the visual question.

### 5.1 Structural Creation

Structural creation networks progressively add or modify visible parts. These are the clearest cases for the tool because the highlighted region usually has a local construction history. The representative sequence can then be read as a compact visual buildup: early states introduce coarse supports, later states refine shape, and the slider gives the artist a spatial handle for scrubbing through those states. We show this with the airplane queried from multiple locations in Figure 7 and a decorated wall with layered ornament patterns in Figure 8.

### 5.2 Composition

Artists often author assets by building components in local networks and then combining those components in a larger composition network. A highlighted region may therefore correspond to a local component, a final assembly, or both. Many operations in these networks are transformations that place parts together, so representative states often capture whole parts or intermediate placement guides such as curves and point locators. We show this with an aqueduct network that combines wall modules with a curve-driven layout in Figure 9, an arena seating area in Figure 10, and a column with separate body and head networks in Figure 11.

### 5.3 Textural and Mixed Change

Some dependency graphs create visible change through materials, masks, erosion, noise, or color fields. These examples test whether a visual query can summarize histories where the salient change is textural rather than part-based. We show this with a mountain network that creates a top-down texture map in Figure 12 and a mountain landscape that combines structural and textural change in Figure 13.




Fig. 8. The highlighted wall region contains both large wall surfaces and thin decorative curves, so the representatives include states for the panel, side column, top edge, and ornament strokes. In this network, the artist builds the components in one larger network rather than separating them into subnetworks, and the tool tends to select more finished components rather than only early initialization states.

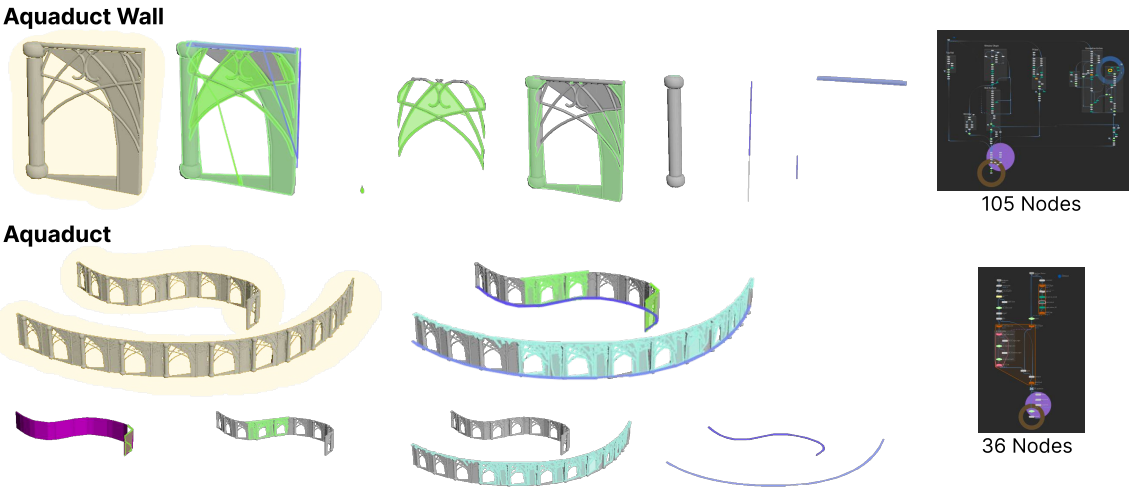



Fig. 9. An aqueduct network places repeated wall modules with a curve-driven layout. Querying one wall module recovers the local column, arch, and brace construction, while querying the full aqueduct returns representatives that include both complete module geometry and curved placement paths. The example shows a composition workflow where the artist may care both about how a module is built and how it is arranged along a curve.

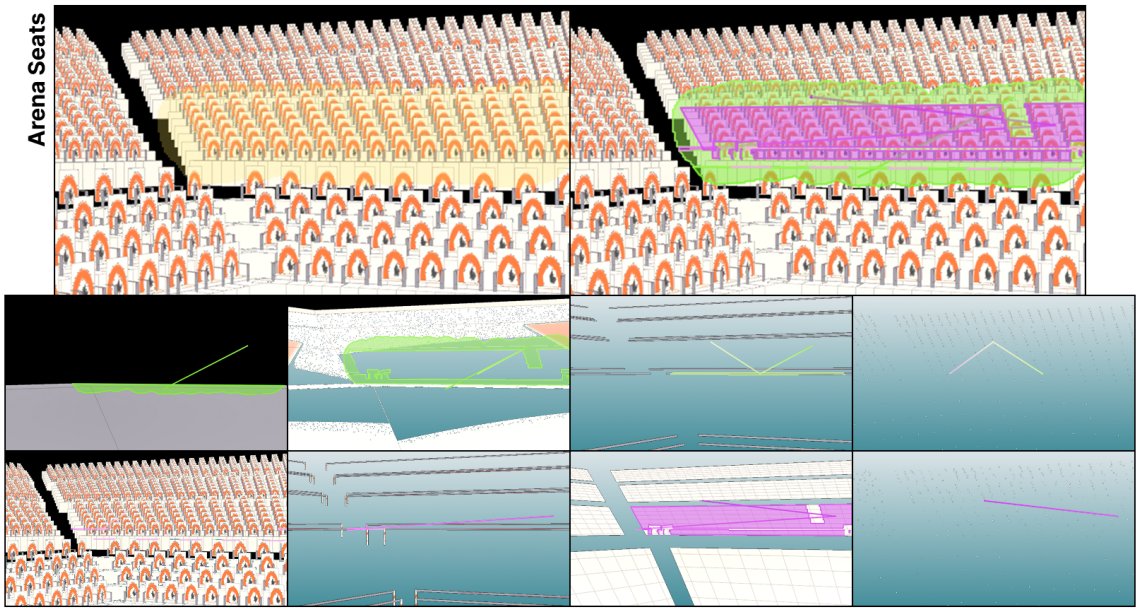



Fig. 10. An arena-seat query shows a repeated-placement network rather than a seat-construction network. Although the artist highlights audience seats, the representatives include floor plates, layout guides, and distant or sparse geometry because the queried image region overlaps several placement-related outputs. This illustrates query ambiguity: the visible intent is "seats," but the dependency-graph evidence also contains nodes that affect where seats are arranged.

## 6 DISCUSSION AND DIRECTION

In-situ dependency-graph understanding can benefit from moving away from the graph interface as the only place where search happens. When artists ask a visual question by drawing directly on the image, they can express the question without first translating it into a node name, parameter name, or dependency-graph region. The in-situ summaries then show a small set of construction-history states near the artifact itself. This lets artists inspect the range of visual effects that contributed to a result without immediately reading node names or tracing nested graph structure.

Our method approaches this problem by summarizing construction history through visual differences and using that summary to create in-situ handles. The image-space formulation is useful because it is mostly representation-agnostic. Construction history of image-based assets such as Photoshop files, video sequences, or image databases can be summarized with the same interaction method. Additionally with AI-assisted content creation, artists may need to understand assets whose construction process is unfamiliar, non-standard, or intentionally obfuscated.

The method is based on a simple premise: image differences can be used as a signal for summarizing construction history. In several examples, the slider handles attach to meaningful visual features such as wing segments, wall textures, and structural supports. The handles also show visible differences between stages of construction, and selecting a tick quickly reveals the corresponding representative state. This gives artists a faster way to find relevant nodes than searching through the graph interface manually, and scrubbing between ticks is more forgiving than repeatedly selecting individual nodes in the graph editor.

Despite this simple interaction, several challenges remain before the tool can be robust enough for everyday artist workflows. The examples suggest that visual difference alone is not specific enough to answer every visual question.

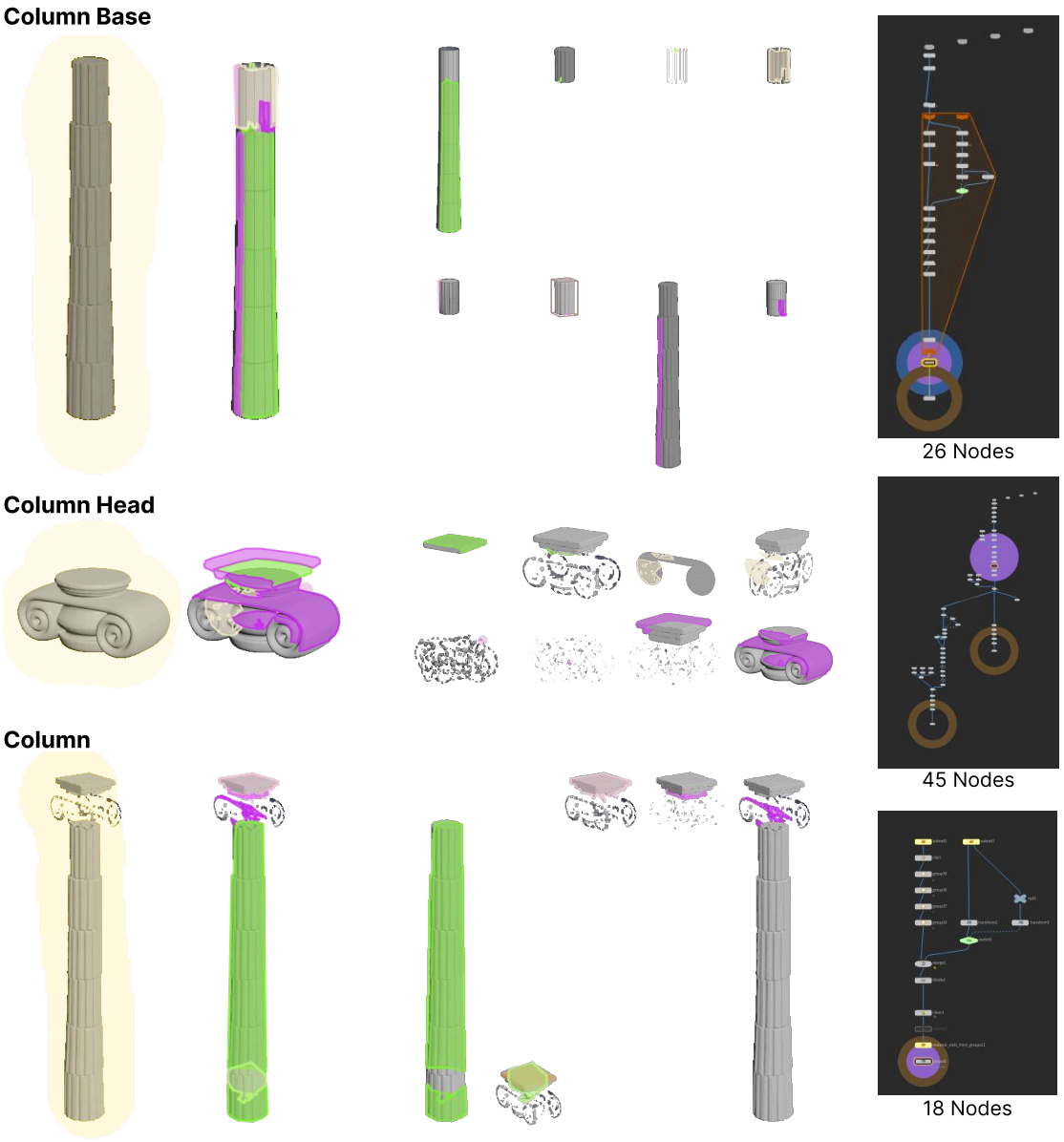



Fig. 11. A column asset contains separate networks for the shaft, the capital, and the assembled column. Querying the base emphasizes a mostly vertical construction sequence, querying the head reveals ornamental scroll and cap geometry, and querying the whole column mixes local component creation with the final composition network. This shows how visual queries can cross from part-building networks into networks that assemble components together.

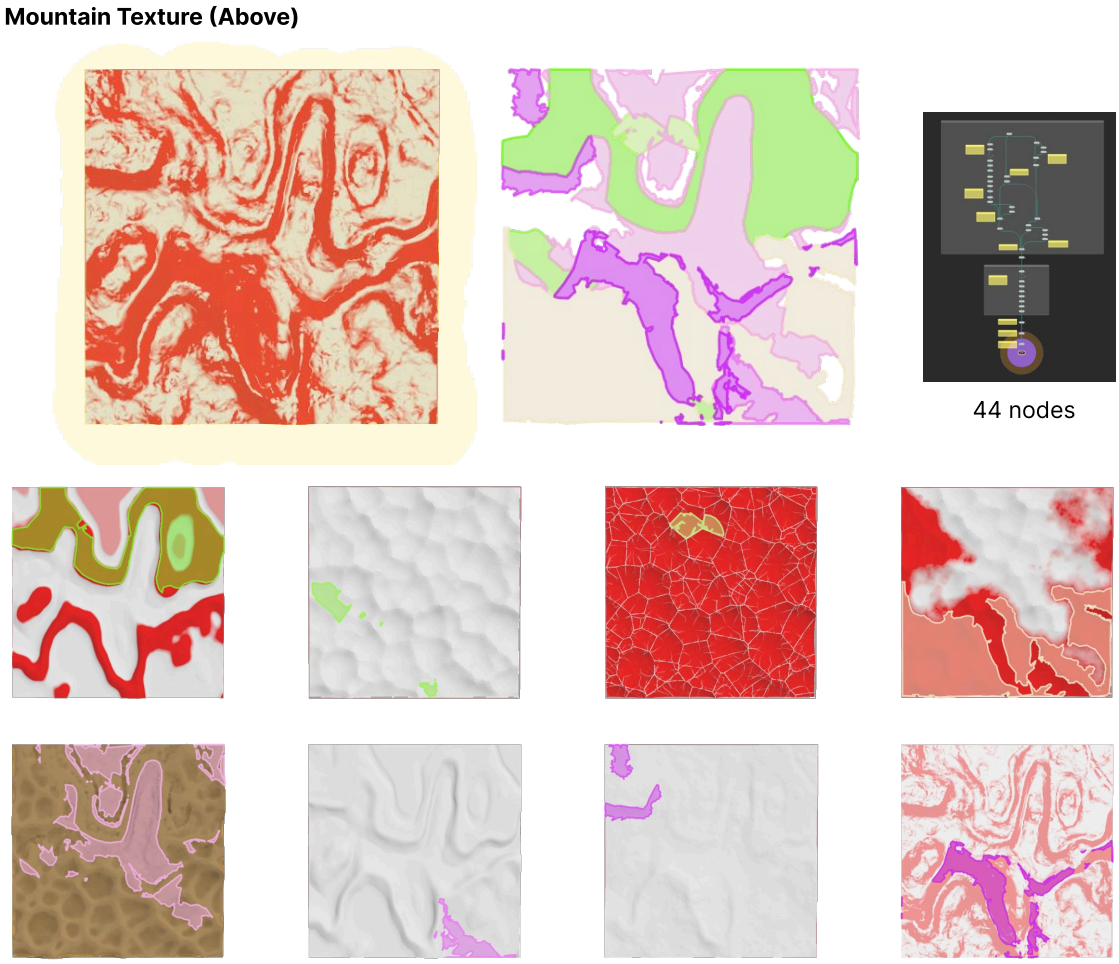



Fig. 12. A top-down mountain-texture query emphasizes material and mask changes more than geometric structure. The representatives show height-field, color, and region-mask states that affect the queried terrain patch. The segments capture important visual features of each representative, but the sequence of segments does not show progressive textural change as clearly as the structural examples. This case shows that the same visual-query pipeline can summarize textural change, but only to a degree.

The tool can reduce a dependency graph into a small set of visually distinct representative states, but whether those states meaningfully summarize the construction history is a separate question.

### 6.1 Query Ambiguity

An artist's visual question is often more specific than a highlighted region of pixels. They may be asking about a particular object, segment, or feature in the image. Highlighting is a quick and natural way to constrain the question, but it is also ambiguous because the highlighted region does not say which object or feature is the intended target.

This ambiguity becomes especially difficult with occlusion. Figure 14 shows a broad arena query where the artist's intent is meaningful but underspecified. The artist highlights the roof of the arena, but because the tool searches for

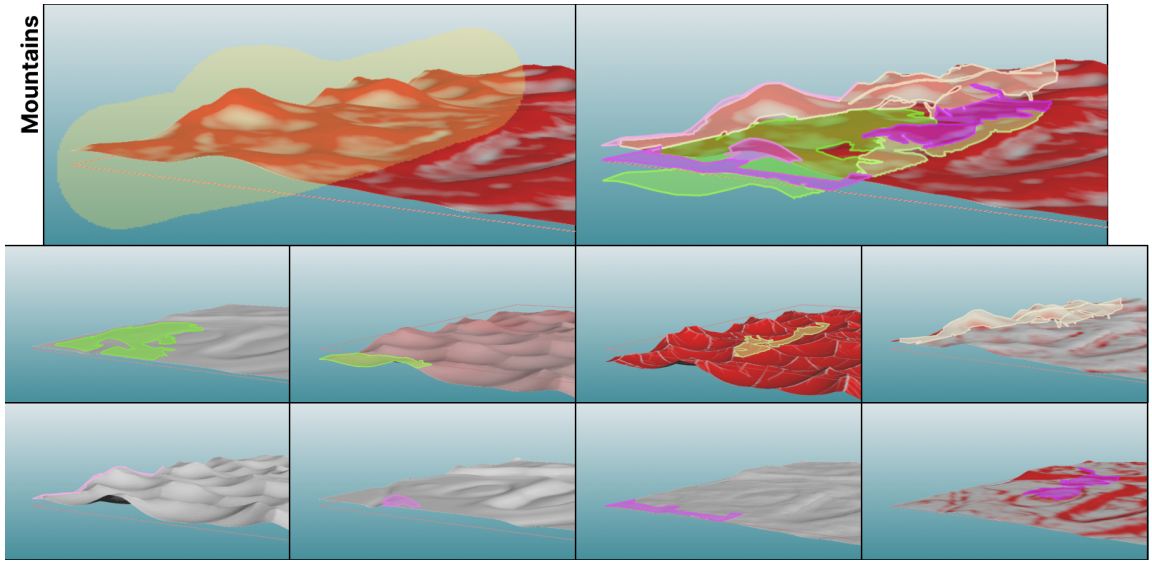



Fig. 13. A mountain landscape combines structural and textural variation. The highlighted terrain region returns representatives for coarse terrain form, surface color, ridges, and later texture masks. Unlike the top-down texture query, this view contains both silhouette and material evidence, so the slider summarizes a mixed history of shape and appearance. When structural and textural operations are interleaved, the segment shapes can be harder to interpret. A clearer separation between structural and textural changes may produce a more readable summary.

visual difference inside the query region, operations behind or near the roof can also enter the result. In Figure 15, the artist highlights a side billboard, but interior seats, roof elements, and hidden outputs still contribute evidence in the same screen-space area. The highlight suggests a question about the roof or billboard, but the method does not yet understand that object-level intent.

Asking the artist to perfectly outline only the exact pixels that correspond to their question is not reasonable. A stronger query model would combine the mask with object identity, depth, and visibility constraints. For example, the initial query could bind to the visible surface under the brush and reject outputs that change behind that surface or on a different object segment. This would better match how artists think about queries: they often mean “this seat block” or “this wall panel,” not every rendered thing under the same pixels.

Even with object identity, composition workflows make the problem harder. As shown in Section 5.2, artists may intentionally ask about how an object is composed rather than how one visible part is constructed. These graphs often transform, instance, and merge geometry across subnetworks, so object identity may not remain stable across the whole construction history. Instead of treating the mask as a fixed constraint for the entire query, a future system may need to interpret the initial highlight as a question about an object or segment, then follow that object or segment through the graph.

### 6.2 Representative Selection and Summary

The current tool clusters nodes by visual similarity and selects representatives from each cluster. This can produce a summary that is visually distinct but not always meaningful. Ideally, a dependency-graph summary should capture important moments in the construction history of the artifact. However, those moments are not always the most visually

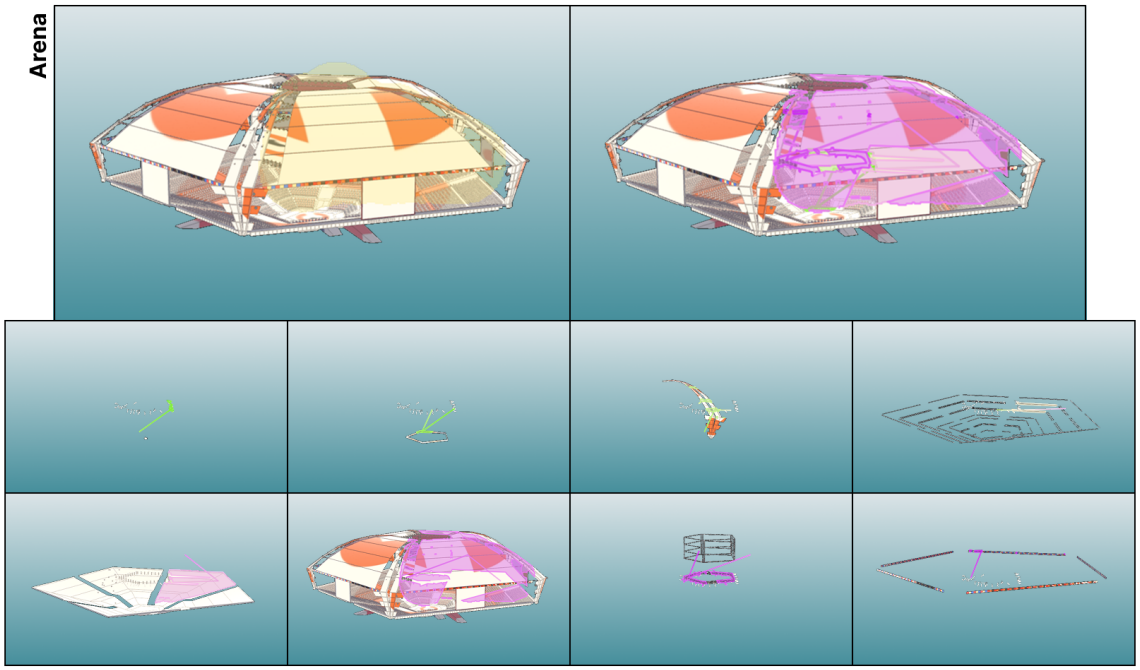



Fig. 14. A broad arena query produces an ambiguous summary. The highlighted roof and seating region returns representatives that include roof panels, construction guides, seating-related traces, and floor or outline geometry rather than visually distinct operations applied only to the roof.

distinct states. Many dependency graphs are organized as long paths of operations with important checkpoints around branches, merges, and transformations. Those operational checkpoints may be better summary points than purely visual clusters.

This suggests that artists may not only want a summary of visual difference. They may want to see the key operational paths through the dependency graph. The airplane example in Figure 7 supports this idea: the airplane is built along a longer path with few branches, and the slider reads as a gradual buildup of the plane body. In contrast, the decorated wall in Figure 8 contains many branches. Its slider contains distinct segments, but the relationship between those segments is less visually continuous, which may make the construction history harder to follow. A future system could apply the slider interaction for intra-cluster navigation instead of inter-cluster selection.

### 6.3 View-Dependent Sliders

For 3D environments, the in-situ slider is attached to scene geometry rather than drawn as a fixed overlay. This keeps the control connected to the artifact during camera navigation, but it also means readability depends on the camera. Because the optimization only considers the initial view, the slider can become less readable from other angles, as shown in Figures 16 and 17. This is a general issue for in-situ methods where controls are attached spatially to the scene.

The segment handles can also obstruct the artifact. Depending on the query intention, an artist may want to see more of the artifact and less of the slider. Reducing the segments to point handles would clear up the view, but it would

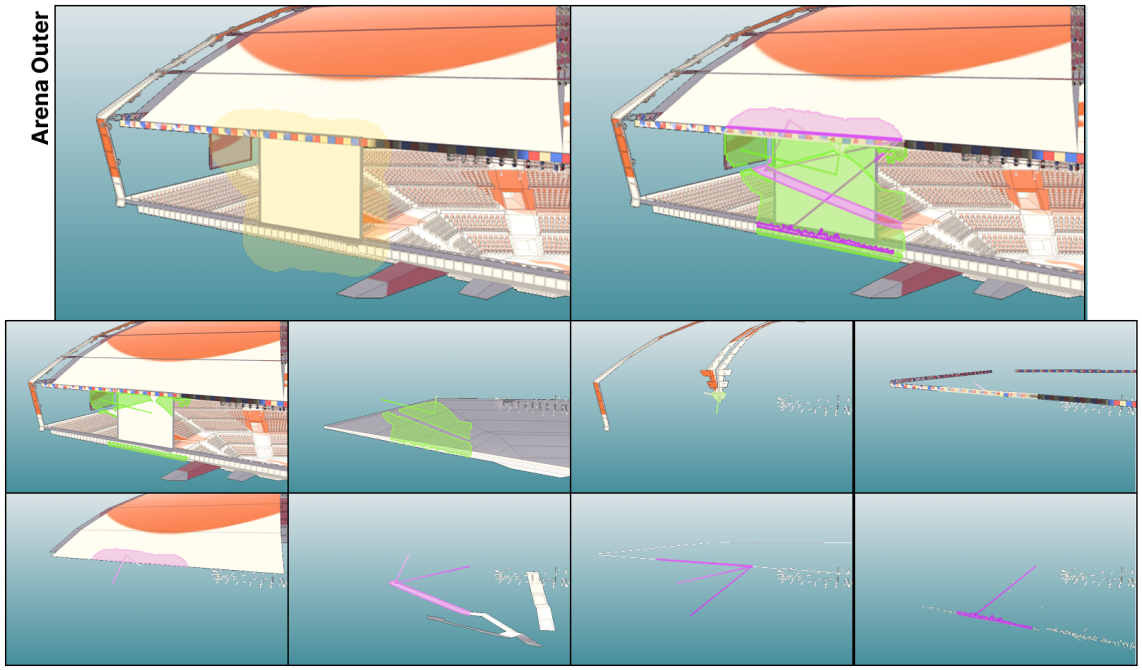



Fig. 15. An outer-arena query shows the effect of occlusion and view ambiguity. The artist highlights a visible side region, but the masked image also overlaps interior seats, roof elements, and hidden construction outputs in the same screen-space area. Because the current method does not reason deeply about object identity or occlusion, unrelated or partially occluded nodes appear in the summary.

also weaken the visual connection between a tick and the representative state it summarizes. Future work could explore more minimal handle designs that still communicate visual difference with less clutter.

### 6.4 Future Direction

The larger goal is a full in-situ dependency-graph workflow. The current system supports in-situ querying, reduction, and navigation, but it stops before in-situ editing and deeper exploration. After artists find a relevant representative state, they still need to return to the graph interface and parameter editor to make changes. A natural next step is to expose selected node parameters or subgraph controls directly near the queried region. Parameters could be mapped to in-situ controls such as sliders, dials, or toggles, but large parameter sets would require selective or summarized controls to avoid adding more visual clutter.

Finally, a formal user evaluation remains necessary. The examples support a qualitative claim that output-centric querying is a plausible entry point into understanding large dependency graphs; this claim comes from the usage by the researchers of the work and feedback from colleagues. They do not yet show when artists are faster, more accurate, or more confident than with graph-based search alone. A comparison study can be implemented that measures how artists find node parameters responsible for a visual effect using the standard Houdini network editor and our tool. However, this work proposes a new interaction paradigm focusing on the benefits of a graph-less interface for an underlying graph-structure. This contrary approach that deviates from the established comfort and standard of graph interfaces

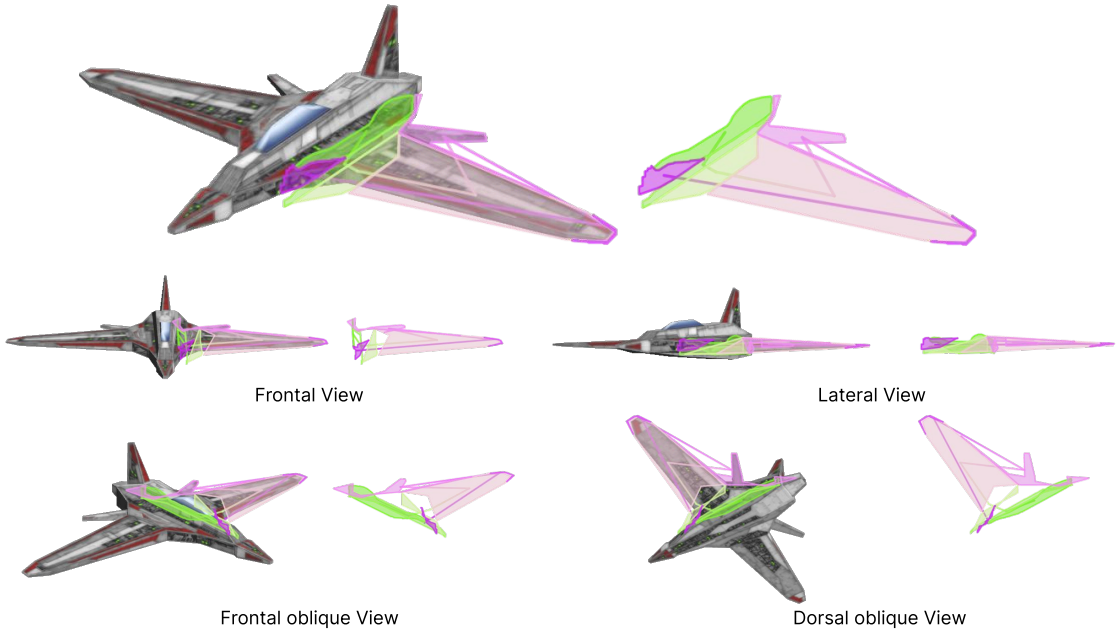



Fig. 16. The same airplane slider viewed from several camera directions. The recovered representatives remain tied to the wing history, but the projected slider shape changes with viewpoint. Some views overlap the segment handles, while side or oblique views compress them into a thin path. This shows that scene-attached sliders suffer from view dependence.

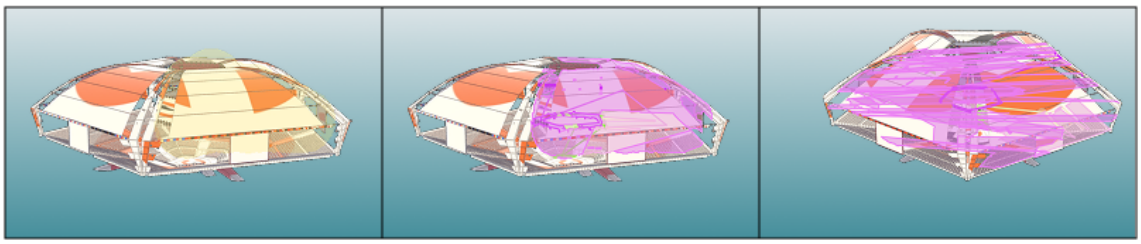


Fig. 17. An arena slider after a view change. The segment handles become difficult to interpret when the camera moves to a view where the selected supports overlap.

presents opportunities to better record whether graph-less workflows reveal surprising benefits that conventional methods lack from traditions. Our work proposes such benefits informally but must be ascertained through more formal evaluation.

## 7 CONCLUSION

We present an in-situ visual-query workflow for exploring dependency graphs from visible consequence rather than from graph interfaces. The artist begins by highlighting a region of interest in the rendered result. The system then captures intermediate node outputs, reduces them into a small representative sequence, and turns that sequence into an

in-situ slider attached to the artifact. This makes construction-history exploration begin from the visual question that motivated the search.

The examples show that this framing is useful across several kinds of procedural networks, but also that these networks behave differently. For structural creation networks, the slider can read as a local buildup of visible parts. For composition networks, the returned representatives may include whole modules, placement guides, curves, and transformations. For textural and mixed networks, the method can recover meaningful visual features.

Our discussion suggests that our tool produces reasonable and simple results for certain dependency graphs by succeeding in choosing visually contrasting node outputs and in-situ UI handles that resemble what's seen. However, our tool requires more context within the visual query to correctly assume the subject being investigated. Additionally, the clustering can fail to select meaningful node operations that summarize important steps throughout the construction history because it relies on an image-based difference assumption.

Future iterations of this work suggest understanding what visually meaningful outputs are to artists and more intelligent parsing of the visual query to match artist intention. However, the method itself provides a plausible alternative for addressing visual questions and exploration outside of traditional, panel-constrained interaction.

## REFERENCES


[1] Radhakrishna Achanta, Appu Shaji, Kevin Smith, Aurelien Lucchi, Pascal Fua, and Sabine Susstrunk. 2012. SLIC Superpixels Compared to State-of-the-Art Superpixel Methods. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 34, 11 (2012), 2274–2282. doi:10.1109/TPAMI.2012.120

[2] Christopher Ahlberg, Christopher Williamson, and Ben Shneiderman. 1992. Dynamic Queries for Information Exploration: An Implementation and Evaluation. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '92)*. ACM, 619–626. doi:10.1145/142750.143082

[3] Eric A. Bier, Maureen C. Stone, Kenneth Pier, William Buxton, and Tony DeRose. 1993. Toolglass and Magic Lenses: The See-Through Interface. In *Proceedings of the 20th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '93)*. ACM, 73–80. doi:10.1145/166117.166126

[4] Steven P. Callahan, Juliana Freire, Emanuele Santos, Carlos E. Scheidegger, Cláudio T. Silva, and Huy T. Vo. 2006. VisTrails: Visualization Meets Data Management. In *Proceedings of the 2006 ACM SIGMOD International Conference on Management of Data (SIGMOD '06)*. ACM, 745–747. doi:10.1145/1142473.1142574

[5] Chad Carson, Megan Thomas, Serge Belongie, Joseph M. Hellerstein, and Jitendra Malik. 1999. Blobworld: A System for Region-Based Image Indexing and Retrieval. In *Visual Information and Information Systems*. Springer, 509–516. doi:10.1007/3-540-48762-X_63

[6] Siddhartha Chaudhuri and Vladlen Koltun. 2010. Data-Driven Suggestions for Creativity Support in 3D Modeling. *ACM Transactions on Graphics* 29, 6, Article 183 (2010), 10 pages. doi:10.1145/1882261.1866205

[7] Andy Cockburn, Amy Karlson, and Benjamin B. Bederson. 2008. A Review of Overview+Detail, Zooming, and Focus+Context Interfaces. *Comput. Surveys* 41, 1 (2008), 1–31. doi:10.1145/1456650.1456652

[8] Robert L. Cook. 1984. Shade Trees. In *Proceedings of the 11th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '84)*. ACM, 223–231. doi:10.1145/800031.808602

[9] Navneet Dalal and Bill Triggs. 2005. Histograms of Oriented Gradients for Human Detection. In *Proceedings of the 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*. 886–893. doi:10.1109/CVPR.2005.177

[10] Jack B. Dennis and David P. Misunas. 1974. A Preliminary Architecture for a Basic Data-Flow Processor. In *Proceedings of the 2nd Annual Symposium on Computer Architecture*. ACM, 126–132. doi:10.1145/800123.803913

[11] Jeanne Ferrante, Karl J. Ottenstein, and Joe D. Warren. 1987. The Program Dependence Graph and Its Use in Optimization. *ACM Transactions on Programming Languages and Systems* 9, 3 (1987), 319–349. doi:10.1145/24039.24041

[12] Myron Flickner, Harpreet Sawhney, Wayne Niblack, Jonathan Ashley, Qian Huang, Byron Dom, Monika Gorkani, Jim Hafner, Denis Lee, Dragutin Petkovic, David Steele, and Peter Yanker. 1995. Query by Image and Video Content: The QBIC System. *Computer* 28, 9 (1995), 23–32. doi:10.1109/2.410146

[13] Paul Haeberli. 1988. ConMan: A Visual Programming Language for Interactive Graphics. In *Proceedings of the 15th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '88)*. ACM, 103–111. doi:10.1145/54852.378490

[14] Mark Hamilton, Zhoutong Zhang, Bharath Hariharan, Noah Snavely, and William T. Freeman. 2022. Unsupervised Semantic Segmentation by Distilling Feature Correspondences. In *International Conference on Learning Representations*. https://openreview.net/forum?id=SaKO6z6Hl0c

[15] Ivan Herman, Guy Melançon, and M. Scott Marshall. 2000. Graph Visualization and Navigation in Information Visualization: A Survey. *IEEE Transactions on Visualization and Computer Graphics* 6, 1 (2000), 24–43. doi:10.1109/2945.841119

[16] Harry Hochheiser and Ben Shneiderman. 2004. Dynamic Query Tools for Time Series Data Sets: Timebox Widgets for Interactive Exploration. *Information Visualization* 3, 1 (2004), 1–18. doi:10.1057/palgrave.ivs.9500061

[17] Takeo Igarashi, Satoshi Matsuoka, and Hidehiko Tanaka. 1999. Teddy: A Sketching Interface for 3D Freeform Design. In *Proceedings of the 26th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '99)*. ACM Press/Addison-Wesley Publishing Co., 409–416. doi:10.1145/311535.311602

[18] Takeo Igarashi, Tomer Moscovich, and John F. Hughes. 2005. As-Rigid-As-Possible Shape Manipulation. *ACM Transactions on Graphics* 24, 3 (2005), 1134–1141. doi:10.1145/1073204.1073323

[19] International Commission on Illumination. 2004. Colorimetry, 3rd Edition. CIE Publication 15:2004.

[20] Joonho Kim and Karan Singh. 2021. Optimizing UI Layouts for Deformable Face-Rig Manipulation. *ACM Transactions on Graphics* (2021). doi:10.1145/3450626.3459842

[21] Vladimir G. Kim, Wilmot Li, Niloy J. Mitra, Stephen DiVerdi, and Thomas Funkhouser. 2012. Exploring Collections of 3D Models Using Fuzzy Correspondences. *ACM Transactions on Graphics* 31, 4 (2012), 54:1–54:11. doi:10.1145/2185520.2185550

[22] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollar, and Ross Girshick. 2023. Segment Anything. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 4015–4026. doi:10.1109/ICCV51070.2023.00371

[23] Vladimir Kolmogorov and Ramin Zabih. 2004. What Energy Functions Can Be Minimized via Graph Cuts? *IEEE Transactions on Pattern Analysis and Machine Intelligence* 26, 2 (2004), 147–159. doi:10.1109/TPAMI.2004.1262177

[24] Bongshin Lee, Catherine Plaisant, Cynthia Sims Parr, Jean-Daniel Fekete, and Nathalie Henry. 2006. Task Taxonomy for Graph Visualization. In *Proceedings of the Working Conference on Advanced Visual Interfaces (AVI '06)*. ACM, 81–85. doi:10.1145/1133265.1133288

[25] Juyong Lee, Brian C. Weymouth, M. C. Lin, and Dinesh Manocha. 2011. ShadowDraw: Real-Time User Guidance for Freehand Drawing. *ACM Transactions on Graphics* 30, 4, Article 27 (2011), 10 pages. doi:10.1145/2010324.1964921

[26] James MacQueen. 1967. Some Methods for Classification and Analysis of Multivariate Observations. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* 1 (1967), 281–297.

[27] Joe Marks, Bradford Andalman, Paul A. Beardsley, William Freeman, Sarah Gibson, Jessica Hodgins, Torin Kang, Brian Mirtich, Hanspeter Pfister, William Ruml, Kyuin Ryall, John Seims, and Stuart Shieber. 1997. Design Galleries: A General Approach to Setting Parameters for Computer Graphics and Animation. *Proceedings of the 24th Annual Conference on Computer Graphics and Interactive Techniques* (1997), 389–400. doi:10.1145/258734.258887

[28] Elie Michel and Tamy Boubekeur. 2021. DAG Amendment for Inverse Control of Parametric Shapes. *ACM Transactions on Graphics* 40, 4 (2021), 173:1–173:14. doi:10.1145/3450626.3459823

[29] Brad A. Myers. 1990. Taxonomies of Visual Programming and Program Visualization. *Journal of Visual Languages & Computing* 1, 1 (1990), 97–123. doi:10.1016/S1045-926X(05)80036-9

[30] Maks Ovsjanikov, Wilmot Li, Leonidas J. Guibas, and Niloy J. Mitra. 2011. Exploration of Continuous Variability in Collections of 3D Shapes. *ACM Transactions on Graphics* 30, 4, Article 33 (2011), 10 pages. doi:10.1145/2010324.1964928

[31] Patrick Paczkowski, Min H. Kim, Yann Morvan, Julie Dorsey, Holly Rushmeier, and Carol O'Sullivan. 2011. Insitu: Sketching Architectural Designs in Context. *ACM Transactions on Graphics* 30, 6, Article 182 (2011), 10 pages. doi:10.1145/2070781.2024206

[32] Karran Pandey, Fanny Chevalier, and Karan Singh. 2023. Juxtaform: Interactive Visual Summarization for Exploratory Shape Design. *ACM Transactions on Graphics* 42, 4, Article 52 (2023), 14 pages. doi:10.1145/3592436

[33] Przemyslaw Prusinkiewicz and Aristid Lindenmayer. 1990. *The Algorithmic Beauty of Plants*. Springer-Verlag.

[34] Peter J. Rousseeuw. 1987. Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis. *J. Comput. Appl. Math.* 20 (1987), 53–65. doi:10.1016/0377-0427(87)90125-7

[35] Ben Shneiderman. 1983. Direct Manipulation: A Step Beyond Programming Languages. *Computer* 16, 8 (1983), 57–69. doi:10.1109/MC.1983.1654471

[36] Tarique Siddiqui, Paul Luh, Zesheng Wang, Karrie Karahalios, and Aditya Parameswaran. 2020. ShapeSearch: A Flexible and Efficient System for Shape-Based Exploration of Trendlines. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD '20)*. Association for Computing Machinery, 51–65. doi:10.1145/3318464.3389722

[37] John R. Smith and Shih-Fu Chang. 1996. VisualSEEk: A Fully Automated Content-Based Image Query System. In *Proceedings of the Fourth ACM International Conference on Multimedia (MULTIMEDIA '96)*. Association for Computing Machinery, 87–98. doi:10.1145/244130.244151

[38] Paul S. Strauss, Rikk Carey, et al. 1992. *An Object-Oriented 3D Graphics Toolkit*. ACM SIGGRAPH Course Notes. Open Inventor.

[39] William Robert Sutherland. 1966. *The On-Line Graphical Specification of Computer Procedures*. Ph. D. Dissertation. Massachusetts Institute of Technology.

[40] Joe H. Ward and Margaret E. Hook. 1963. Hierarchical Grouping to Optimize an Objective Function. *J. Amer. Statist. Assoc.* 58, 301 (1963), 236–244.

[41] Colin Ware and Robert Bobrow. 2005. Supporting Visual Queries on Medium-Sized Node-Link Diagrams. *Information Visualization* 4, 1 (2005), 49–58. doi:10.1057/palgrave.ivs.9500090

[42] Jun-Yan Zhu, Yong Jae Lee, and Alexei A. Efros. 2014. AverageExplorer: Interactive Exploration and Alignment of Visual Data Collections. *ACM Transactions on Graphics* 33, 4, Article 160 (2014), 11 pages. doi:10.1145/2601097.2601145